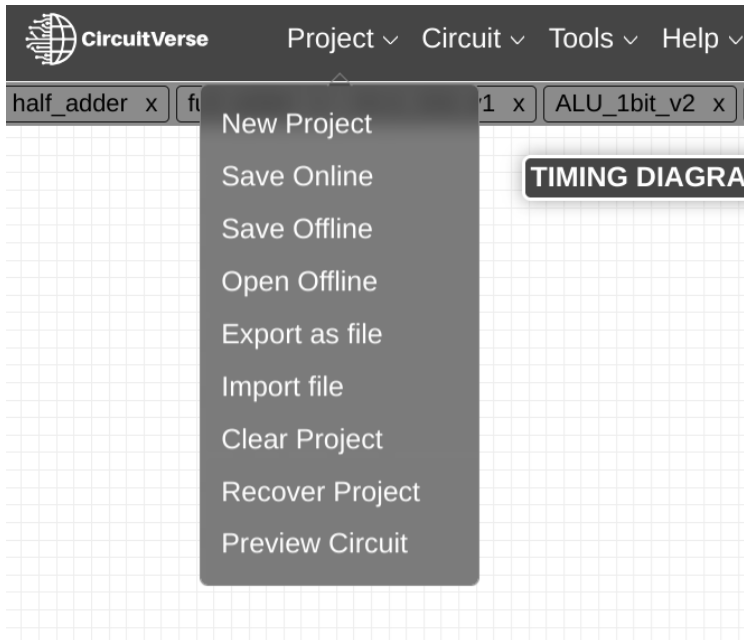


Lab 10

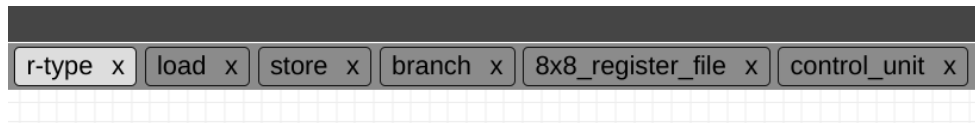
Il Processore RISC-V

Un processore RISC-V semplificato

- Scaricare il template da Moodle (RISCV_template.cv)
- Importare questo template su CircuitVerse (Import file)
- Completare il progetto del processore "RISC-TO"



Da completare



Esercizio: "RISC-TO"

- "RISC-TO" - scomodissimo e inesistente processore RISC-V a 8 bit
- Supporta solo alcune istruzioni: **lb**, **sb**, **beq**, **add**
(e forse anche alcune altre istruzioni del Tipo-R mai testate)
- Ha 256 B di memoria per le istruzioni e 256 B per i dati
- Un singolo programma caricato in memoria EPROM
(purtroppo non è un boot loader o un sistema operativo)
- Solo 8 registri (x0..., x7)
- Usa lo stesso formato delle istruzioni del RISC-V (32 bit per istruzione)

- Mancano alcuni componenti nel processore RISC-TO
- **Completare:**
 - L'Unità di Controllo Centrale
 - Il Register File (Lab 9)

"RISC-TO" - Unità di Controllo

r-type x

load x

store x

branch x

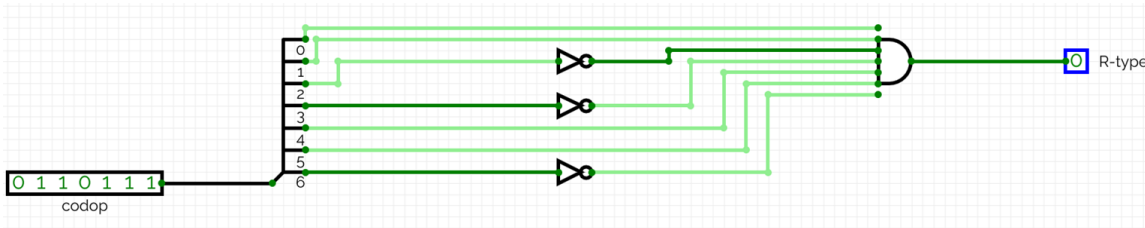
8x8_register_file x

control_unit x

- L'Unità di Controllo Centrale:

- r-type - dato il **codop**, ritorna 1 se è una istruzione del **Tipo-R**
- load - dato il **codop**, ritorna 1 se è una **load**
- ...
- control_unit - usa i sottocircuiti precedenti per programmare il datapath

- Esempio:



Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

"RISC-TO"

Il programma caricato in memoria

Assembly

```
    lb  x1, 0(x0)
    lb  x2, 1(x0)
    lb  x3, 2(x0)
    add x4, x1, x2
    beq x4, x3, else
    sb  x4, 0(x0)
    beq x0, x0 end

else:
    add x1, x1, x1
    sb  x1, 0(x0)
end:
    beq x0, x0 end
```

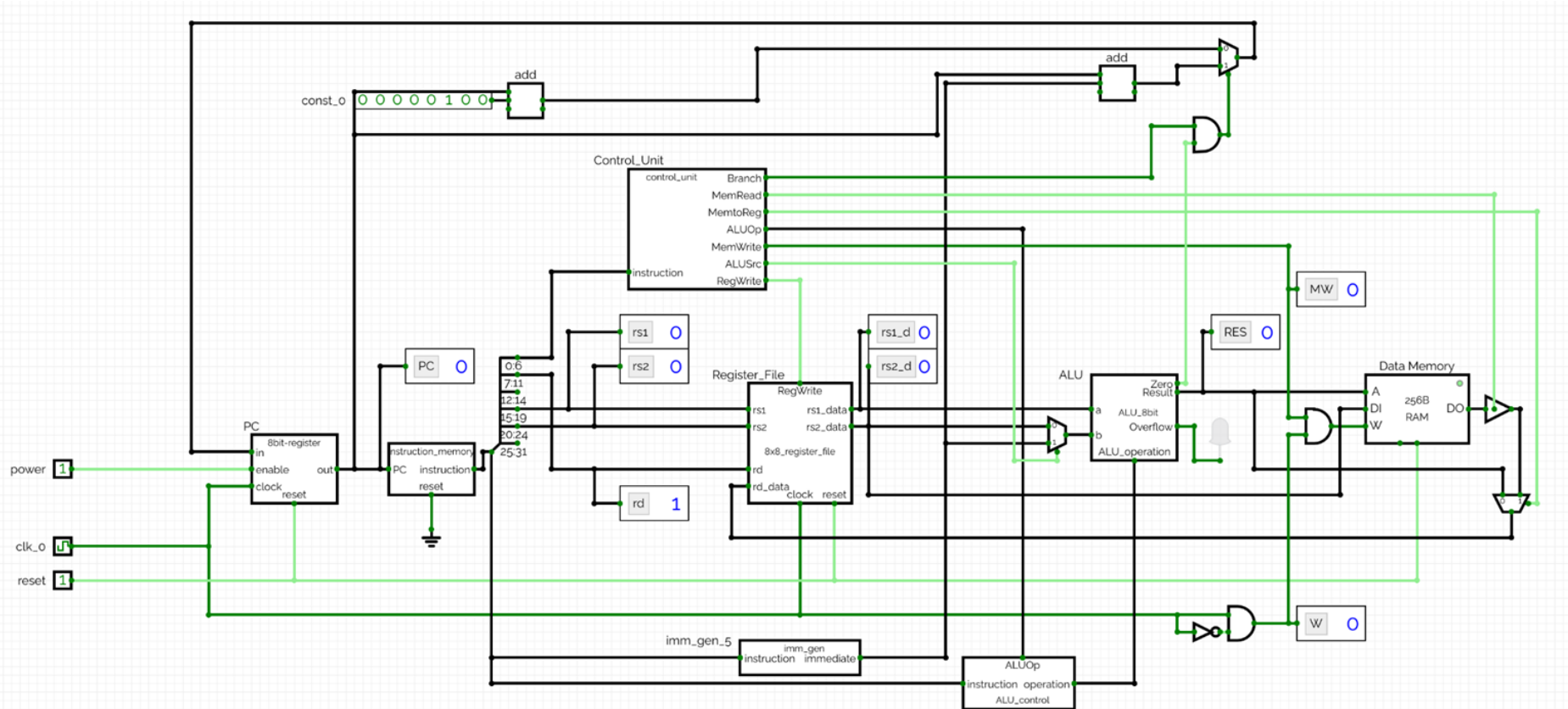
ISA

```
00000000 000000 000000 000 00001 0000011
00000000 00001 00000 000 00010 0000011
00000000 00010 00000 000 00011 0000011
00000000 00010 00001 000 00100 0110011
00000000 00011 00100 000 01100 1100011
00000000 00100 00000 000 00000 0100011
00000000 00000 00000 000 01100 1100011
00000000 00001 00001 000 00001 0110011
00000000 00001 00000 000 00000 0100011
00000000 00000 00000 000 00000 1100011
```

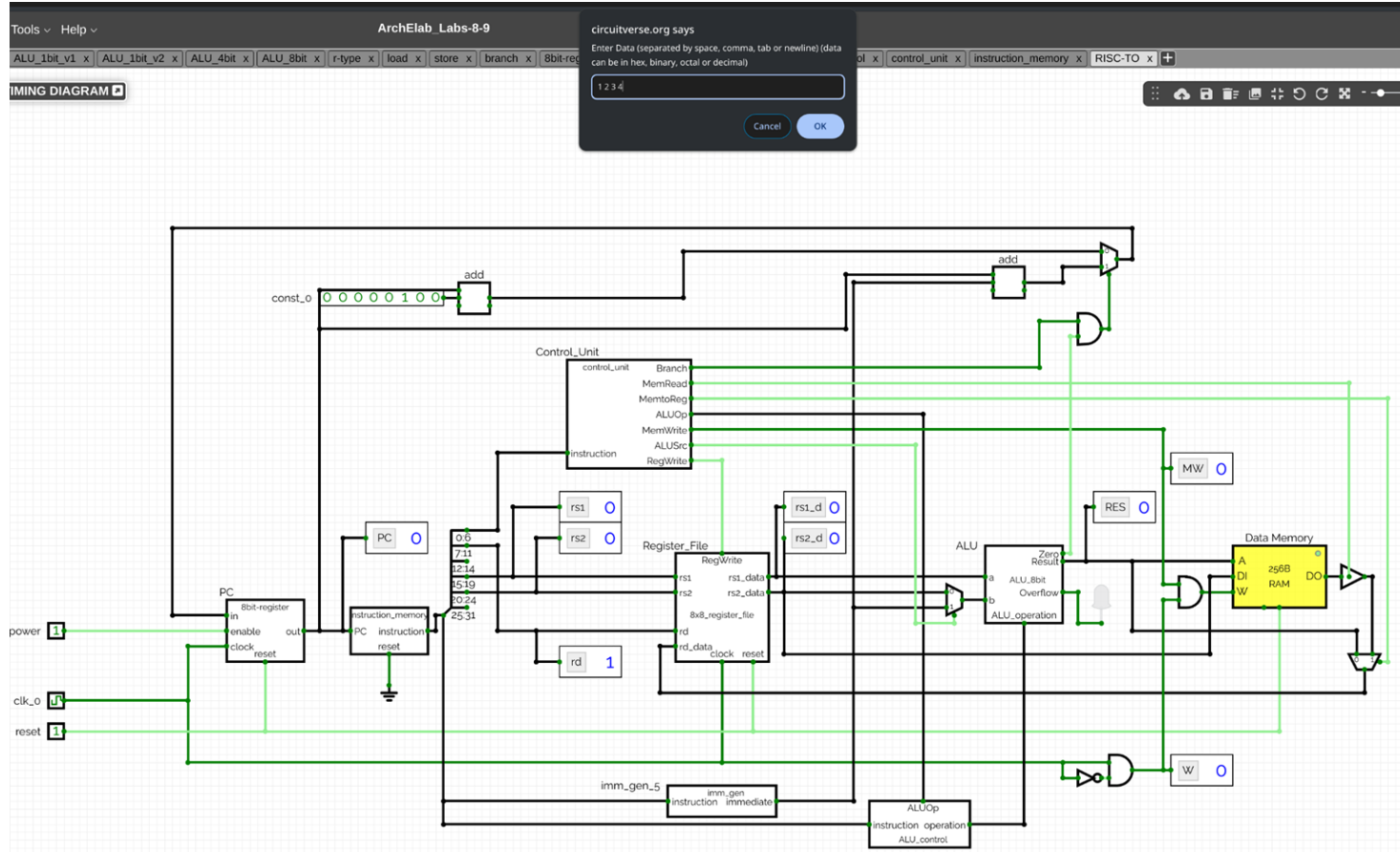
Text Segment

Address	Code
00	0x00000083 lb x1,0(x0)
04	0x00100103 lb x2,1(x0)
08	0x00200183 lb x3,2(x0)
0c	0x00208233 add x4,x1,x2
10	0x00320663 beq x4,x3,12
14	0x00400023 sb x4,0(x0)
18	0x00000663 beq x0,x0,12
1c	0x001080b3 add x1,x1,x1
20	0x00100023 sb x1,0(x0)
24	0x00000063 beq x0,x0,0

reset = 1: azzera il PC, i registri e la memoria dati (ma non cancella il programma)



doppio click sulla memoria per caricare dati nella memoria
esempio: $M[0] = 1$; $M[1] = 2$; $M[2] = 3$; $M[3] = 4$



Core Dump: stampa i valori in memoria nella console del browser

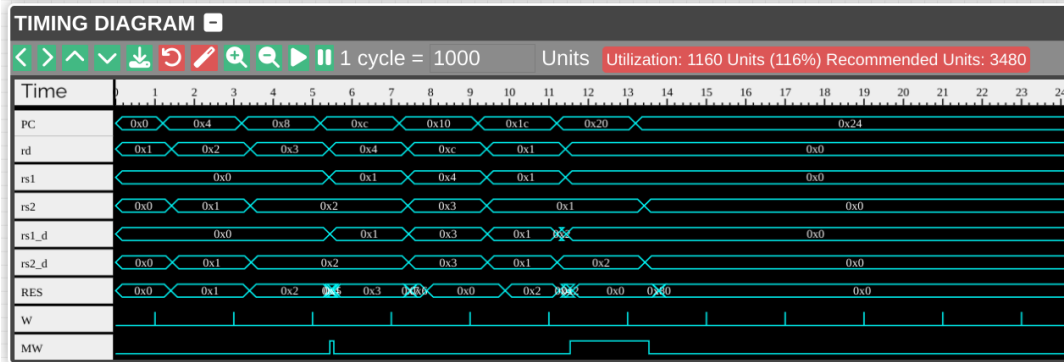
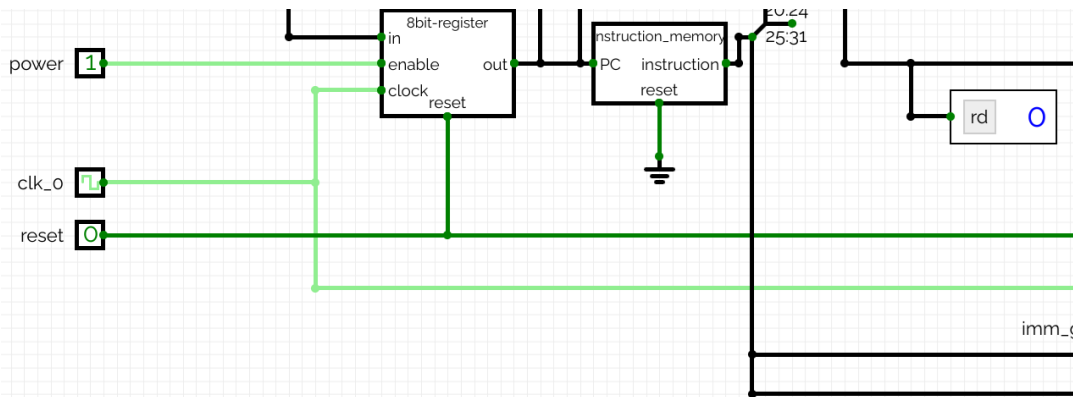
The image shows a web browser interface with a dark theme. On the left, a panel titled "Idilio Drago" contains a "RISC-TO" tab and a "PROPERTIES" section. The "PROPERTIES" section has a "RAM" subsection with the following settings:

- BitWidth: 8
- Delay: 0
- Label: Data Memory
- Label Direction: UP
- Address Width: 8

Below these settings are three buttons: "Core Dump", "Load Data", and "Reset".

On the right, the browser's developer console is open, showing a log of messages. A red arrow points to the "Data Memory" section of the log, which displays a large array of memory values, mostly null, with some numerical values at the beginning. The console also shows a message about a 'dblclick' handler taking 3932ms.

Risultato atteso della simulazione



```
[Violation] 'dblclick' handler took 4330ms
```

simulator-dd93e5d0.js:487

▼ Data Memory

simulator-dd93e5d0.js:329

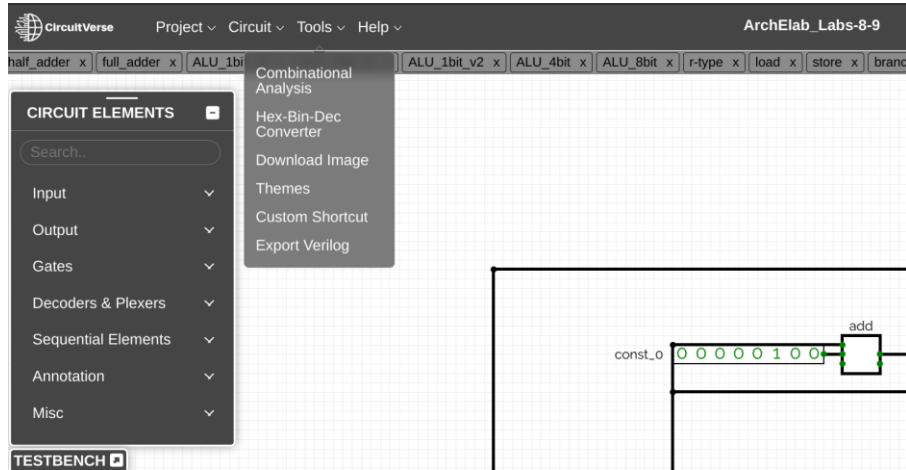
[illegible]

▼ Data Memory

```
simulator-dd93e5d0.js:329
```

[illegible]

Consegna



Esportare il file Verilog
Consegnare su Moodle
Il progetto potrà essere argomento
dell'esame orale